

中国科学院大学
University of Chinese Academy of Sciences

研究生学位论文中期报告

报告题目面向硬件验证符号执行的路径探索优化技术研究

学生姓名 梁佳辉 学号 202118015029022

指导教师 杨秋松 职称 研究员

学位类别 工学博士

学科专业 计算机科学与技术

研究方向 形式化方法

研究所（院系） 中国科学院软件研究所

填表日期 2026 年 7 月

中国科学院大学制

填 表 说 明

1. 本表内容须真实、完整、准确。
2. “学位类别”名称：学术型学位填写哲学博士、教育学博士、理学博士、工学博士、农学博士、医学博士、管理学博士，哲学硕士、经济学硕士、法学硕士、教育学硕士、文学硕士、理学硕士、工学硕士、农学硕士、医学硕士、管理学硕士等；专业学位填写工程博士、工程硕士、工商管理硕士（MBA）、应用统计硕士、翻译硕士、应用心理硕士、农业推广硕士、工程管理硕士、药学硕士等。
3. “学科专业”名称：学术型学位填写“二级学科”全称；专业学位填写“培养领域”全称。

报告提纲

1. 选题背景与意义 2

2. 国内外本学科领域的发展与趋势 6

2.1 符号执行技术与动态验证的结合 6

2.2 硬件符号执行引擎 6

2.3 基于依赖分析的剪枝策略 7

2.4 大模型辅助的符号执行验证方法 7

3. 主要研究内容与预期目标 8

3.1 研究内容 8

3.1.1 单周期基于独立分支的路径探索策略 9

3.1.2 多周期基于值预测的路径探索策略 10

3.1.3 大模型辅助的基于里程碑的路径探索策略 10

3.2 预期目标 11

4. 课题研究进展 12

4.1 基于独立分支的路径探索策略 12

4.2 多周期基于值预测的路径探索策略 18

4.3 里程碑引导的多周期 RTL 符号执行验证方法 20

5. 研究工作计划与进度安排 25

6. 已取得研究成果列表 26

7. 参考文献 26

1.选题背景与意义

随着集成电路工艺进入纳米尺度及系统复杂度呈指数级增长，确保硬件设计的功能正确性（Functional Correctness） 和安全性（Security） 已成为芯片设计流程中至关重要且成本高昂的环节。当前主流的硬件验证方法主要包括：

1. 基于仿真的动态验证（Dynamic Simulation-Based Verification）
2. 形式化验证（Formal Verification）：

基于仿真的动态验证使用大量的测试向量（Test Vectors） 或随机生成的输入激励，在仿真器（如 ModelSim, VCS, Questa）上运行硬件模型（RTL 或 Gate-level），观察输出是否符合预期。通常辅以功能覆盖率（Functional Coverage）和代码覆盖率（Code Coverage） 来衡量测试充分性。该方法的优势是技术成熟且工具链完善，可应用于大规模设计，直观易用。但是囿于覆盖率瓶颈，难以穷尽所有可能的输入组合和状态序列，因此深层次状态和边界情况（Corner Cases）并不会被覆盖，可能导致关键缺陷（Critical Bugs）或安全漏洞（如硬件木马）被遗漏，这需要设计架构和微架构领域的专家设计特定的测试用例对这些场景进行覆盖。而且测试向量生成效率低下，验证周期长，成本高昂。

形式化验证使用数学方法严格证明设计满足其形式化规约（Formal Specification）（如属性 Property）。主要技术包括：

- 模型检测（Model Checking）： 系统性地穷举所有可能状态，验证属性是否在所有状态下成立。
- 定理证明（Theorem Proving）： 使用逻辑推理和数学证明来验证设计正确性。
- 等价性检查（Equivalence Checking）：验证两个设计（如 RTL vs Gate-level）在功能上等价。

完备性（Completeness）是形式化方法的最大优势，理论上可以证明设计 100% 满足属性（在给定资源限制内），无需测试向量，擅长发现深层次错误。但随着验证对象日趋复杂的设计，导致验证过程中出现状态空间爆炸（State Space Explosion）的问题。对复杂设计，状态空间可能大到无法在有限时间和资源内完成验证。且验证规约编写难度高，需要专家编写精确的属性，人力成本高且易出错。此外，可扩展性（Scalability）对大型设计（如现代 CPU/SoC）仍是巨大挑战。

符号执行技术本质上属于形式化验证，核心思想是将输入抽象为符号（Symbols），同时探索多条执行路径，通过求解路径约束（Path Constraints） 的可满足性（Satisfiability）来生成覆盖该路径的测试输入或发现不可达路径/违反属性的情况，算法一描述了具体流程。

算法 1 The algorithm of traditional symbolic execution

```
Input: Program P;
Output: Test input set Γ;
Γ ← ∅;
Let inst_init be the initial instructions;
mem_init ← ∅;
s_0 ←< true, inst_init, mem_init >; the initial state
worklist ← s_0;
while (worklist ≠ ∅)
    < pcon, inst, mem >← selectState(worklist);
    for (each successor inst' of inst)
      if (inst is abort or halt)
        τ ← solve(pcon)
        Γ: = Γ ∪ τ
      continue
     else if (inst is branch if(c))
        if (pcon ∧ c is UNSAT under mem)
         continue
        s' ←< pcon ∧ c, inst'mem >
        else if (inst is v: = exp)
          s' ←< pcon, inst', mem[v ← exp] >
        endif
        worklist ← worklist ∪ s'};
     endfor
end while
return Γ;
```

模型检测在验证过程中是直接状态空间穷举的方式，在多时钟周期场景下极易产生路径爆炸问题。符号执行技术的优势相较于模型检测的直接状态空间穷举，通过路径约束求解以及特定的路径剪枝策略，能避免探索冗余的路径，更有效地处理由长序列输入触发的复杂状态，这在发现硬件安全漏洞（如侧信道、特权升

级）和潜伏木马方面尤为关键。此外，符号执行技术还提供了比仿真更高的覆盖率潜力。

虽然符号执行技术有上述优势，但在硬件验证的应用中，仍存在以下问题：

一．硬件设计固有的高复杂性问题。

随着硬件复杂性的日益提升，仍然会带来路径爆炸问题。此路径爆炸问题主要来自两方面：

1. 单周期内路径爆炸

单周期内的路径爆炸主要来自于硬件本身设计的复杂度：随着硬件设计中分支数目N_{br}的提升，执行路径数目会成指数增长至$2^{N_{br}}$。

2. 多周期路径爆炸

在对硬件进行验证的过程中，大多数场景需要极大的验证深度才能触发。简单来说，验证周期数为 t 才能触发的验证状态，需要在路径空间大小为$2^{N_{br}\times t}$中去探索满足验证属性的路径。

二．符号执行引擎对硬件的特性支持问题。

符号执行技术目前在软件分析与验证领域的应用要强于硬件验证领域，其核心瓶颈在于软硬件之间存在本质的语言语义差异与工具链不匹配，具体体现在以下两个方面：

1. 软硬件语言与执行机制的差异

软件程序代码（如 C/C++）通常是基于控制流的顺序执行逻辑，而硬件描述语言（HDL，如 Verilog/VHDL）本质上描述的是具有高度并行性的寄存器传输级（RTL）结构或门级网表。目前许多将软件分析技术应用于硬件验证的工作，通常需要先将硬件设计翻译为软件模型（例如转化为 C++模型）以便分析。这种跨语言的转换会导致严重的抽象层级不匹配（Abstraction Mismatch）。在这一翻译过程中，真实硬件设计中固有的寄存器计算、底层信号交互以及高度并发性等原生硬件行为特征会发生丢失。这使得符号执行引擎在处理这些软件抽象模型时，往往会产生偏离真实硬件运行状态的“语义漂移”，无法精确建模硬件状态的转移过程。

2. 软硬件验证工具链与评估指标的不匹配

传统的软件符号执行引擎及其工具链主要依赖控制流结构，如指令（instruction）、基本块（basic block）、条件分支（conditional branch）等进行路径探索和覆盖率

反馈。然而，硬件验证工具链关注的指标与软件截然不同。虽然软件中的条件分支在一定程度上可以映射为硬件中的多路选择器（Mux）信号，但传统软件引擎无法感知并捕获硬件验证中至关重要的特有覆盖率指标，例如信号翻转（Signal Toggling）、有限状态机（FSM）的复杂状态转换以及底层模块的激活等。由于缺乏对硬件原生工具链指标的支持，传统的符号执行引擎不仅无法准确反馈硬件状态空间的探索进度，也无法通过约束求解有效地将路径探索引导至特定的硬件目标状态（Target States），从而极大地限制了其在复杂硬件验证中的效率与定向探索能力。

因此在硬件验证工作中，符号执行技术的应用仍有很大的优化空间，比如利用路径剪枝，约束求解等方法，降低冗余路径，优化约束规模，从而提升符号执行技术在硬件验证中的效率与可扩展性，提升硬件验证的效率。本研究主要针对单周期和多周期中指数级增长的路径空间，在已有的软硬件符号执行引擎上，结合硬件结构特征，程序静态分析或智能化辅助手段（如大模型引导）进行定向探索，从而大幅削减冗余路径探索，从而降低约束求解的规模与开销。

2.国内外本学科领域的发展与趋势

符号执行技术在软硬件验证领域都有着较为广泛的应用，虽然本研究致力于在硬件验证领域，但软件验证领域的相关技术也应作为参考，这些技术主要包括路径探索中通过静态分析等方法进行的路径剪枝策略，以及结合大模型辅助的相关验证框架。

2.1 符号执行技术与动态验证的结合

符号执行技术在硬件验证领域结合点较多：符号执行技术通过与动态验证相结合，提高了验证覆盖率，或用于硬件的安全性验证，致力于发掘漏洞或木马。

作为符号执行和具体测试的结合体，混合执行（Concolic Testing），使用具体值（Concrete Values）替代部分符号变量（Symbolic Variables），以避免分析不必要的路径。QUEBS[10]通过限制分支（Branch）可被选择的次数来防止路径爆炸。Lyu 和 Mishra [12] 提出了一种方法，将有向测试生成（Directed Test Generation）问题映射为目标搜索（Target Search）问题，并利用学习和聚类技术进行搜索，以减少重叠搜索。Meng 和 Kundu [12] 生成了用于识别片上系统（System on Chip）中关键漏洞（Critical Exploits）的安全测试用例。Ahmed [13] 和 Farahmandi 在多进程多周期环境中通过静态测量目标距离（Target Distance），并利用距离反馈（Distance Feedback）有效引导搜索，从而实现了高总体覆盖率。

符号执行也与模糊测试（Fuzzing）技术结合，以在不进行繁重约束求解的情况下快速枚举输入。Dai 和 Yavuz [14] 采用模糊测试和静态分析来生成可疑硬件元素的模型，这些模型用作预言机（Oracles）来指导符号执行。Jingxuan He 和 Gishor Sivanrupan 等人 [15] 使用 LEARCH 生成的测试作为初始模糊测试种子（Initial Fuzzing Seeds），使流行的模糊测试器 AFL[21] 能够发现更多路径和安全违规（Security Violation）。具体来说，LEARCH 是他们提出的一种基于学习（Learning-based）的策略，通过选择有前景的状态（Promising States）来应对路径爆炸问题。

2.2 硬件符号执行引擎

上述许多工作首先将硬件设计转换为以 C/C++ 代码表示的软件模型，然后使用面向软件验证的符号执行工具 KLEE[5]。从硬件到软件模型的转换可以通过商业 EDA 工具（例如 Cadence, Synopsys）或开源工具（例如 Verilator [22], Icarus Verilog [23]）来完成。

一些工作直接在 Verilog 上进行符号执行。 RTL-ConTest [12] 直接在 SoC RTL 上操作，生成了用于识别片上系统关键漏洞的安全测试用例。他们使用 Python 分析原始 Verilog 以生成控制流图（CFG）来辅助路径选择。Sylvia[2] 利用 pyVerilog 生成 Verilog 的抽象语法树（AST）以识别独立的 always 块（Independent Always Blocks）。它通过分别探索独立的块来减少需要探索的冗余路径。SEFI [20] 将 Sylvia 作为框架，实现了一种用于信息流验证（Information Flow Verification）的方法和搜索启发式（Search Heuristics）。

2.3 基于依赖分析的剪枝策略

一些软件验证方面的工作提出了基于依赖分析（Dependence Analysis）的剪枝策略。 CBC[34] 提出了兼容分支集（Compatible Branch Set, CBrSet）的概念，通过在探索过程中动态计算 CBrSet，可以在不降低分支覆盖率（Branch Coverage）的情况下剪枝更多冗余路径。DSGE 提出了一种称为交互依赖（Interactive Dependency）的新依赖关系，它在探索路径以剪枝冗余路径的同时维护依赖关系。[24] 通过将输入划分为非干扰块（Non-interfering Blocks）来避免探索冗余路径。[25] 关注输出（Outputs），如果两条路径相对于程序输出具有相同的相关切片（Relevant Slice），则将它们放在同一分区（Partition）中。相关切片是动态数据和控制依赖关系（Dynamic Data and Control Dependencies）的传递闭包（Transitive Closure），也包括涉及那些通过未执行而影响输出的语句的潜在依赖关系（Potential Dependencies）。

2.4 大模型辅助的符号执行验证方法

近年来，大语言模型（LLM）凭借其强大的代码表征、逻辑推理和自然语言处理能力，逐渐被引入形式化验证与符号执行领域，成为突破传统方法在面对复杂环境建模、结构化输入求解以及长周期状态探索时所遭遇的路径爆炸与求解瓶颈的重要研究趋势。在通用软件符号执行方面，国内外学者广泛探索了利用大模型

提取先验知识、优化约束求解及生成高质量初始种子的新型验证范式，例如 LangSym 框架通过理解用户手册自动构建外部文件环境并生成种子以指导 KLEE 引擎探索，CONCOLLMIC 利用大模型智能体（Agent）使用自然语言推理复杂的路径约束以减轻传统求解器的计算压力，而针对具有高度结构化输入的程序，COTTONTAIL 提出了由大模型驱动的“求解-补全”新范式，利用思维链（CoT）生成高覆盖率的输入种子。此外，LLM 强大的文本理解能力也被成功应用于特定领域的规则提取与合规性审计，如 SymGPT 能够从标准文档中自动提取自然语言规则并转化为严格的程序约束，从而自动化检测智能合约中是否存在违反规范的漏洞或后门；更有前沿工作（如 Feng 等人的评估）开始探索大模型直接阅读 C 代码来模拟符号执行输出并预测受约束最深执行路径的理论潜力，虽然当前其直接模拟的准确率仅约为 20%，但证明了其在未来分担重度计算环节或提供启发式指导方面的巨大价值。最为关键的是，在现代硬件寄存器传输级（RTL）验证领域，结合大模型克服多周期时序路径爆炸已成为当前的前沿热点，Tan 等人提出利用大模型分析 RTL 静态结构与仿真数据以提供突变建议，并结合 LoRA 技术与专属合成语料库对模型进行硬件语义微调。

3.主要研究内容与预期目标

3.1 研究内容

尽管目前硬件符号执行引擎 Sylvia 直接对 RTL 进行解析，相较于传统使用软件符号执行引擎对硬件进行验证的方法，极大地减少了 query 次数，缓解了路径爆炸问题，但仍有极大的优化空间：其只是利用 Verilog 中彼此独立的 always 块之间的独立性分别进行路径探索，这仅仅减少了重复的 query 次数，实际上探索的状态和路径总量并未减少。硬件描述语言仍有区别于软件的独有特点，仍然可以利用这些特点在现有引擎基础上进行优化。此外，对于软硬件语言共有的特性，亦可借鉴软件符号执行已有的成熟思想进一步优化。在此基础上，本研究首先在架构层面优化了符号执行引擎中 case 语句的处理机制，停止对 case 结构进行简单的内联展开，而是将 match-guard 约束直接引入到路径条件（Path

Condition）中，这有效避免了底层控制流图（CFG）机制被绕过的基础性寻径缺陷，为后续的复杂路径探索奠定了坚实的正确性基础。

本研究综合利用硬件语言的特性与大语言模型（LLM）的先进推理能力，在单周期、多周期以及全局高层语义引导三个维度全面优化了路径探索策略。

首先，通过分析阻塞赋值声明顺序与执行顺序一致且当周期立即生效的特点，本研究探究了组合逻辑中分支依赖对可达状态的影响，设计了单周期基于独立分支的路径探索策略，有效避免了对已访问状态和冗余路径的重复探索。

其次，针对非阻塞赋值在未来几周期内必定生效的特性，本研究设计了多周期基于值预测的路径探索策略，通过预测相关变量值并提前进行约束求解，在实际执行到对应周期前即可提前剪枝矛盾路径。

最后，针对多周期复杂状态机中深层漏洞难以触发的问题，本研究引入了大模型技术，提出了大模型辅助的基于里程碑的路径探索策略。该策略利用 LLM 自动生成高层语义里程碑来动态引导底层的路径选择，并结合容错裁决机制减缓并迭代修正大模型固有的“幻觉”误导，从而在宏观时间跨度上进一步破解了时序路径爆炸的验证瓶颈。

3.1.1单周期基于独立分支的路径探索策略

在硬件验证过程中，传统方式是通过将硬件描述语言表示的设计转换成 C/C++ 表示的软件模型，再通过传统软件模型引擎进行符号执行，这存在两大缺陷：硬件到软件的转换可能引入错误，忽略硬件并行特性导致路径重复探索。当前最新工具 Sylvia 虽然通过独立 always 块并行探索减少路径重复（降低 SMT 求解器调用次数）。尽管极大地减少了验证时 query 的不必要开销，但仅解决跨模块的路径冗余。本研究分析后发现单个 always 块内关于组合逻辑电路的执行路径仍存在冗余：存在多条路径导向的最终状态是相同的，Sylvia 并不能对当前路径是否导向已探索状态，引入了额外的运行时开销。

本研究首先对 RTL 描述的电路进行分析，发现没有电路连接的多路选择器彼此并不会影响输出信号的取值，同时本研究发现彼此独立的多路选择器之间的选择信号也是独立的，选择信号对应的就是 RTL 描述的分支表达式结果。基于此发现，本研究引入了 RTL 中独立分支集合的概念，设计并实现了独立分支集

合的计算方法，该计算方法核心是对每个组合逻辑模块进行依赖性分析，通过基本的数据依赖和控制依赖，借助依赖关系的传递性确定完全独立的分支集合。这一计算过程在路径探索之前进行。在路径探索过程中，本研究根据已得到的独立分支集合，提出了对探索路径进行剪枝的优化方法，在辅助硬件符号执行过程中避免访问导向已访问过状态的冗余路径，进一步缓解可能存在的路径爆炸问题，缩短符号执行的验证时间。

3.1.2 多周期基于值预测的路径探索策略

在硬件验证过程中，传统方式是通过将硬件描述语言表示的设计转换成 C/C++ 表示的软件模型，再通过传统软件模型引擎进行符号执行，这存在两大缺陷：硬件到软件的转换可能引入错误，忽略硬件并行特性导致路径重复探索。当前最新工具 Sylvia 虽然通过独立 always 块并行探索减少路径重复（降低 SMT 求解器调用次数）。尽管极大地减少了验证时 query 的不必要开销，但对于多周期的路径探索，仍未进行进一步的路径优化处理。本研究分析得出，在多周期探索过程中，单周期路径个数为 m，周期数为 N，则总探索路径数目为 m 的 N 次方，验证复杂设计时，路径爆炸问题是难以避免的。

根据对硬件描述语言的分析，本研究发现可以利用非阻塞赋值在指定周期后生效的特性对多周期的符号执行进行优化。利用此特性，可以推断出对应变量在多个周期后的确定的值，基于此可以对后面周期涉及变量的值进行预测，将预测的值结合时钟作为路径约束集合并入当前路径约束集合，提前求解约束的满足性，该约束集合是矛盾的，则提前避免了探索矛盾路径，减少了路径探索开销。首先，本研究设计并实现了对于非阻塞赋值的解析算法，确定非阻塞赋值涉及的信号，对此信号进行依赖性分析，确定依赖于此信号的其他信号。根据此分析，本研究设计了信号值的预测算法，获得相关信号在对应周期数后的表达式，将此表达式集合并入路径约束集合，判断路径约束的可满足性。本研究通过提前进行路径约束的可满足性判断，优化了探索冗余路径的总体开销，进一步提升了符号执行的验证效率。

3.1.3 大模型辅助的基于里程碑的路径探索策略

针对多周期 RTL 设计中深层序列漏洞难以触发、传统方法遭遇时间路径爆炸的问题，本部分拟研究一种大模型辅助的里程碑引导符号执行方法（MileSE，

Milestone-Guided Symbolic Execution for Multi-Cycle RTL Verification）。该内容主要关注大语言模型与底层硬件符号执行引擎之间的上下文绑定与双向反馈机制引导的路径探索策略。

首先，研究面向 RTL 硬件描述语言的静态分析与上下文精化技术。通过影响锥（Cone-of-Influence, COI）剪枝，剥离与目标断言无关的硬件逻辑，提取精简的控制流图（CFG），并构建上下文增强提示（Context-Augmented Prompt）。该提示机制将迫使大模型的推理过程与真实的硬件拓扑结构深度绑定，从而生成包含具体意图、时序边界（Cycle Bound）和 SystemVerilog 条件规范的里程碑序列。

其次，研究基于状态反馈的动态重规划（Re-planning）机制。打破传统大模型单次生成的开环模式，当符号执行引擎在某段路径中陷入停滞或发现前置里程碑不可达时，研究如何捕获当前的活动路径约束、寄存器快照和控制流节点，生成状态提示（State Prompt）并反馈给大模型，驱动其根据当前的精确符号状态重新生成可行的后续引导，从而实现闭环的探索优化。

3.2 预期目标

本研究面向符号执行应用于硬件验证中的路径探索优化技术，旨在根据硬件描述语言的特性，探索在硬件符号执行引擎上的探索路径优化方法。本研究针对硬件描述语言的阻塞赋值和非阻塞赋值，设计有效的路径探索策略，分别在单周期和多周期两个维度进行路径剪枝，在路径探索过程中避免探索冗余路径，缓解路径爆炸问题。本研究预期达到目标如下：

- 提出一种单周期的路径探索优化方法：定义独立分支并设计分析算法确定每个 always 块内的独立分支集合，在路径探索过程中记录独立分支被探索状态，并结合独立分支集合判断当前路径所导向的叶节点状态是否已经被探索，根据判断结果进行路径剪枝，避免冗余路径的重复探索，提高路径探索效率。
- 提出一种面向多周期的路径探索优化方法：通过分析硬件设计中非阻塞赋值，预测对应周期数后的信号及相关信号的值，提前几周期后的路径

约束可满足性求解，提前剪枝错误路径，避免不必要的路径探索，提高多周期的路径探索效率。

- 提出一种大模型辅助的基于里程碑的路径探索优化方法：通过大模型对硬件设计和验证属性进行分析，给出在路径探索过程中的里程碑约束，设计对应的路径探索策略，在路径探索过程中优先选择原有路径约束合取里程碑约束成立的路径，并提出相应机制，及早发现并修复因大模型幻觉问题生成的里程碑约束不合法而陷入的无效路径探索。

4.课题研究进展

本研究旨在缓解符号执行技术在复杂硬件验证中面临的“路径爆炸”问题。传统的硬件符号执行往往需要先将硬件描述语言（HDL）转化为软件模型，再利用 KLEE 等软件验证领域符号执行验证工具进行分析，这种跨语言转换极易导致硬件原生并发特征的丢失与抽象层级的不匹配。为此，本研究直接以硬件原生的符号执行引擎（Sylvia）为基础平台，分析软硬件描述语言在执行语义上的异同，从而设计高效的路径剪枝与探索引导方法，尽量避免访问大量冗余路径，从而缓解路径爆炸问题。

本课题的研究工作由微观结构到顶层语义逐步深入，主要包含以下三个方面的研究进展：

- 研究内容一： 面向组合逻辑电路，提出了一种基于独立分支分析的单周期路径剪枝与探索策略优化方法；
- 研究内容二： 面向时序逻辑电路，提出了一种基于寄存器值预测的多周期路径剪枝与探索策略优化方法；
- 研究内容三： 结合人工智能前沿技术，提出了一种大语言模型（LLM）辅助的基于里程碑引导的多周期路径探索策略。

4.1 基于独立分支的路径探索策略

本研究提出了一种面向组合逻辑电路的路径剪枝裁剪策略，核心思想是利用组合逻辑电路中的阻塞赋值执行顺序与声明顺序一致的特点，通过分析赋值相关变量之间存在的依赖关系，判断当前执行路径是否是冗余并进行路径裁剪。主要工作流程如图 1 所示。在初始化对符号执行状态等进行初始化后，对硬件设计进行静态的依赖分析，在进行依赖分析前，本研究引入了所涉及的不同种类的依赖定义，以及“独立分支”的概念，以及配套的识别算法。依赖分析后，与传统的符号执行算法流程基本一致，但是在路径探索过程中添加了对当前路径是否冗余的判断。

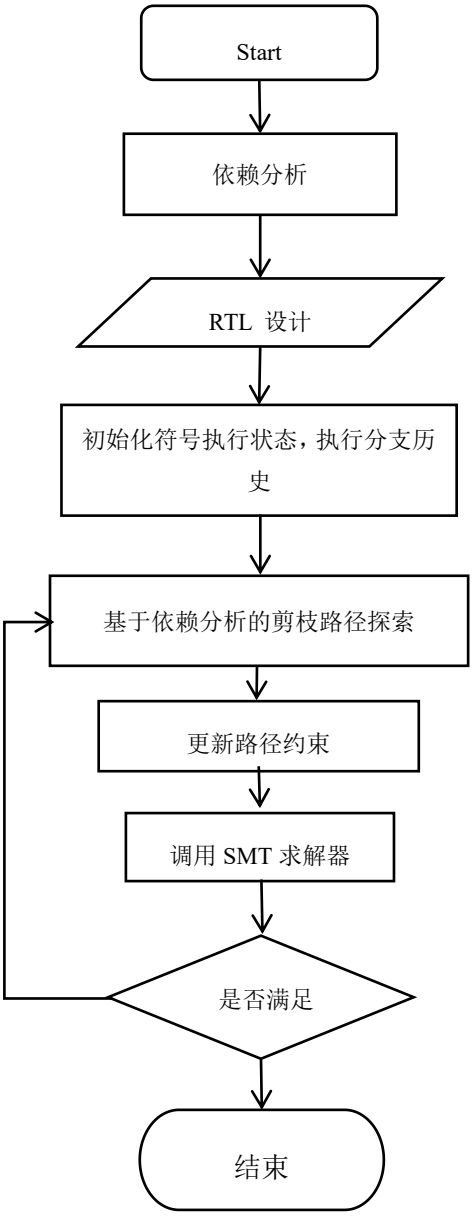


图 1 路径探索流程

4.1.1 依赖关系与独立分支的定义

组合逻辑是一种数字电路，其输出仅由当前输入值决定，而不涉及任何存储或反馈。在组合逻辑中，输出是当前输入值的函数，电路没有内部状态。组合逻辑的行为可以用布尔代数描述，其中输出表示为输入变量的布尔函数。通过评估给定输入值的布尔函数，可以确定组合逻辑电路的输出。在组合逻辑中，赋值是阻塞性赋值，这意味着代码执行顺序与语句顺序相同，并且赋值立即执行。我们可以利用这些特性进行依赖分析，从而剪枝冗余路径并减少搜索空间。

通常，控制流图用于分析程序中语句之间的依赖关系。我们通过定义 1 给出控制流图的定义。

定义 1（控制流图）

控制流图是一个二元组<N, E>其中：

- N 表示程序节点的集合
- E 表示程序节点之间的执行流向

控制流图是一个有向图，表示程序中的执行流程。图中的节点表示程序的基本块，而边则表示这些块之间的执行流。我们用br_i表示程序中第 i 行的条件分支，用n_j表示第 j 行的任何语句。控制流图用于分析程序不同部分之间的依赖关系，如控制依赖和数据依赖。定义 2 与定义 3 分别是控制依赖和数据依赖的定义。

定义 2（控制依赖）

控制依赖是一个映射关系$CD: Br \times N \rightarrow \{T, F\}$，其中：

Br表示一个分支节点，N是非分支节点，当N是否执行依赖于Br分支节点的取值（真或假）时，称N与Br存在控制依赖关系。

定义 3（数据依赖）

数据依赖是一个映射关系$DD: N \times N \rightarrow \{T, F\}$，其中：

N_i与N_j是程序的两个基本块，当N_i与N_j之间存在一条执行路径 π，N_i定义了一个变量v且N_j使用了v，且在执行路径 π 中没有其他基本块修改变量v，则N_i与N_j存在数据依赖关系。

本研究发现，每条执行路径均由分支逻辑（branching logics） 派生而来。当遇到分支语句 if (c) 时，系统会生成两条路径，并更新其路径条件（path condition）：

- 路径 1： $pcon = pcon \wedge c$
- 路径 2： $pcon = pcon \wedge \neg c$

路径的分离（divergence） 本质上是不同分支语句组合作用的结果。因此，必须分析各分支语句间的依赖关系（dependence relations）。然而，控制依赖与数据依赖均无法准确描述这些分支语句间的关联性。我们引入分支依赖描述这种关系，分支依赖定义如下：

定义 4（分支依赖）

分支依赖是关于分支节点和程序节点的依赖关系：当分支节点Br_i与声明节点N_j存在传递性数据依赖或者控制依赖时，称分支节点Br_i 与声明节点N_j之间存在分支依赖关系。

同时，我们为每一个分支节点维护一个分支依赖集合。

定义 5（分支依赖集合）

如果一个集合中的所有元素均与Br_i存在分支依赖关系，则称该集合为Br_i的分支依赖集合。

分支依赖通过计算控制依赖和数据依赖的传递闭包，得出两个分支之间是否存在分支依赖关系。

4.1.2 独立分支集合与其计算

本研究发现，不存在分支依赖关系的两个分支，彼此的执行是互不干扰的，即彼此是独立的。彼此独立的分支，路径探索并不会互相影响。我们可以通过记录这些分支的探索历史以及分支间的独立关系，判断当前分支衍生的路径是否是冗余的。因此本研究引入了独立分支和独立分支集合的概念，定义如下：

定义 6（独立分支）

当分支节点Br_i与Br_j各自的分支依赖集合不存在交集，则Br_i与Br_j互为独立分支。

定义 7（独立分支集合）

当一个集合中的所有元素均与Br_i互为独立分支，则称此集合为Br_i 的独立分支集合

算法 2 给出了计算独立分支集合的算法。该算法主要基于对分支依赖关系的计算。为每个分支维护一个集合，该集合的元素均是与该分支的分支依赖集合相交为空集的分支。基本思想是为每个分支节点遍历其他分支节点，判断两个分支节点的分支依赖集合是否包含相同的元素，如果不包含，该两个节点互为独立分支。

算法 2 Calculation of Independent Branch Set

```
Input: BDSet, AllBranchSet;
Output: IndependentBranchSet;
BranchSet ← AllBranchSet;
for br in AllBranchSet do
   while (BranchSet ≠ ∅) do
      br' = BranchSet[0];
      if (BDSet(br) ∩ BDSet(br') = ∅)
         IndependentBranchSet[br] = IndependentBranchSet[br] ∪ br'}
      endif
      BranchSet = BranchSet \ br
   endwhile
endfor
return IndependentBranchSet;
```

4.1.3 路径探索中的剪枝策略

本研究基于静态分析得到的独立分支集合作为指引，遇到分支是否是冗余路径，即此分支衍生的路径最终达到的状态是否已经被历史路径覆盖。算法 3 给出了判断是否进行裁剪的算法。

算法 3 IfPrune

```
Input: BDSets, AllBranchSet, current state s;
Output: True or False; If the output is True, the path will be pruned
ExecutedBranches ← GetBranch(s.pcon); Function GetBranch analyze s.pcon to get the branches through the current path
for br' in ExecutedBranches do
   if (Br' ∉ IndependentBranchSet[br]) then
      return False;
   end if
end for
if br has been explored then
      return True;
else
```

```
10:     return False;
11: end if
12: return False;
```

该算法通过分析当前路径的分支历史，每个分支的子路径执行历史，以及所有分支的独立分支集合判断当前路径是否可以被裁剪。当且仅当当前访问分支的子路径全被探索过，独立于历史记录中的所有分支时，当前路径可以被裁剪。

4.1.4 路径探索策略的测试与验证

本研究在硬件引擎 Sylvia 上进行开发，以 5 个简单硬件设计以及一个开源处理器核 PICORV32 作为测试用例，与 Sylvia 在探索路径数目，SMT 查询次数上进行对比。表 1 给出了测试用例的结构分析，分别统计了代码行，分支个数以及 always 块的个数。

硬件设计	代码行	分支个数	Always 块个数
PICORV32	2193	230	29
DarkRISCV	531	51	2
Hack@DAC18	96444	4452	883

表 1 测试用例代码结构分析

此外，针对 PICORV32 这一设计，我们还在运行时间和内存消耗上进行对比。根据实验结果，我们发现在 5 个简单硬件设计上效果突出，部分设计路径缩减可达 50%。在 PICORV32 设计上，路径裁剪效果也能达到将近 10%，同时，由于需要记录各分支的历史记录和独立分支集合，本研究所提出的方法也会消耗更多的内存使用。图 5，图 6 以及表 2 分别是本研究方法与 Sylvia 的对比。

方法	探索路径数	SMT 查询次数	运行时间	内存使用
Sylvia	4992	194524	837.83s	130704KB
独立分支引导探索策略	4498	120428	604.37s	150428KB

表 2 PICORV32 上各指标对比

4.2 多周期基于值预测的路径探索策略

在面向硬件描述语言的 Sylvia 框架基础上，本研究从多周期角度优化路径探索策略，缓解 DUT 过于复杂时带来的路径爆炸问题。利用非阻塞赋值在下一时钟周期必定生效的特点，本研究将下一时钟周期可预测的变量值，作为路径约束，提前到当前周期的可满足性检查，如果求解器无法得出解，则说明该条路径下一周期将无效，避免下周期对此路径进行探索，在一定程度上缓解了多周期的路径爆炸问题。同时，为了扩大下一周期的约束，本研究提出了新的依赖关系，借助此依赖关系对下一周期的变量值进行计算。本研究流程与图 2 路径探索流程基本一致，但是在依赖分析步骤后，进入符号执行路径探索阶段后先进行值预测。

4.2.1跨周期依赖与计算

为了在下一周期的变量值预测过程中，更为全面地覆盖更多变量，本研究设计提出了跨周期依赖的定义进行辅助。跨周期依赖的定义如下：

定义 8（跨周期依赖）

跨周期依赖是关于分支节点和声明节点的依赖关系：在阻塞赋值中，当分支节点N_m与声明节点N_j存在传递性数据依赖或者控制依赖时，且Br_i 与N_m 存在非阻塞赋值的传递性数据依赖或控制依赖关系，则分支节点Br_i与声明节点N_j存在跨周期依赖关系

这里有两个核心概念：“循环依赖 (Cycle Dependence) ”和“循环依赖集 (Cycle Dependence Set) ”。这些概念用于分析程序中节点（例如语句或操作）之间的依赖关系，特别关注于阻塞赋值和非阻塞赋值之间潜在的依赖关系，本研究利用这种依赖，结合当前路径的各变量值的状态，推测下一周期可以被确定的变量值。

算法 4 给出了计算各节点之间跨周期依赖关系的算法，通过提供的阻塞赋值依赖和非阻塞赋值间的依赖关系，利用依赖集合的传递闭包，计算出和当前分支节点存在依赖关系的下一周期的节点。

算法 4 Calculation of Cycle Dependence Set

Input: $NonBlockingBDSets, BlockingDepSets, AllBranchSet$;

Output: $CycDSets$;

```
for each br ∈ AllBranchSet do
    for each node ∈ NonBlockingBDSet(br) do
        CycDSet(br) = CycDSet(br) \cup BlockingDepSet(node)
    end for
end for
return CycDSets;
```

4.2.2 路径约束的更新

本研究描述了如何利用上一节提到的跨周期依赖，对下一周期的值进行预测，从而生成新的路径约束，新生成的路径约束与当前的路径约束进行析取后的结果，送给 SMT 求解器进行求解。路径约束的计算如算法 5 所示：

算法 5 Calculation of path constraint of next cycle

Input: $CycDSets$;

Output: pc_{t+1};

```
pc_{t+1} ← True
for each br ∈ AllBranchSet do
    for each node ∈ CycDSet(br) do
        updateMem(node.lhs)
        if node is a branch then
            pc_{t+1} = pc_{t+1} ∧ mem[node]
        end if
    end for
endfor
return pc_{t+1};
```

该算法对 DUT 中每一个分支的跨周期依赖集合进行遍历，根据当前状态对要被赋值的变量值进行更新，如果当前被遍历的节点是一个分支节点，则更新路径约束。

4.2.3 实验设计

本研究仍在 Sylvia 基础上进行开发，仍然以 Sylvia 作为基线进行对比，在探索路径数目，SMT 查询次数，以及发现 bug 能力进行对比。

测试集以开源处理器核为主，包括 PICORV32，OR1200，以及 HACKDAC 竞赛涉及的处理器核。

4.3 里程碑引导的多周期 RTL 符号执行验证方法

本研究工作提出了一种面向多周期寄存器传输级（RTL）硬件设计的里程碑引导符号执行方法（MileSE）。核心思想是利用大语言模型（LLM）生成高层语义里程碑来指导底层的符号执行路径探索，从而解决复杂硬件验证中面临的时间路径爆炸（Temporal Path Explosion）问题。

如图 4-3-1 所示，MileSE 的系统架构将整个验证过程划分为三个主要功能模块及一个裁决器：RTL 预处理器（RTL Pre-processor）、基于 LLM 的里程碑生成器（LLM-Based Milestones Generator）、双轨路径选择模块（Dual-Track-based Path Selection）以及裁决器（Adjudicator）。其具体工作流程如下：

1. RTL 预处理与里程碑生成： 验证过程始于 RTL 预处理器。本研究对 RTL 代码和目标断言进行静态分析，提取结构化的信号依赖关系与控制流约束。这些信息随后与原始 RTL 代码及断言共同注入预定义的提示词模板中，引导大模型生成格式化为 JSON 的里程碑序列。

2. 双轨路径选择： 生成的里程碑随后由双轨路径选择模块与裁决器共同使用。路径选择模块采用双轨评分机制对候选路径进行评估，引导探索过程优先选择最有可能触发漏洞的分支。

3. 基于裁决器的动态容错： 为确保系统能够从由错误里程碑导致的误导探索中安全恢复，裁决器在每个时钟周期主动监控执行状态。通过持续观察执行引擎与硬件状态的实时交互，裁决器强制实施了一套容错策略。具体而言：它利用滑动窗口前瞻（Sliding-Window Lookahead）来绕过局部的时序错位；在全局探索陷入停滞时，触发超时驱动的重规划循环（Timeout-Driven Regeneration Loop），向大模型反馈系统状态摘要以查询新的里程碑；此外，这种持续监控支持了急切目标评估（Eager Target Evaluation），即在每一步执行中检查全局断言条件，确保无论是否已满足所有中间里程碑，都能在漏洞变得可达的准确瞬间将其精准捕获。

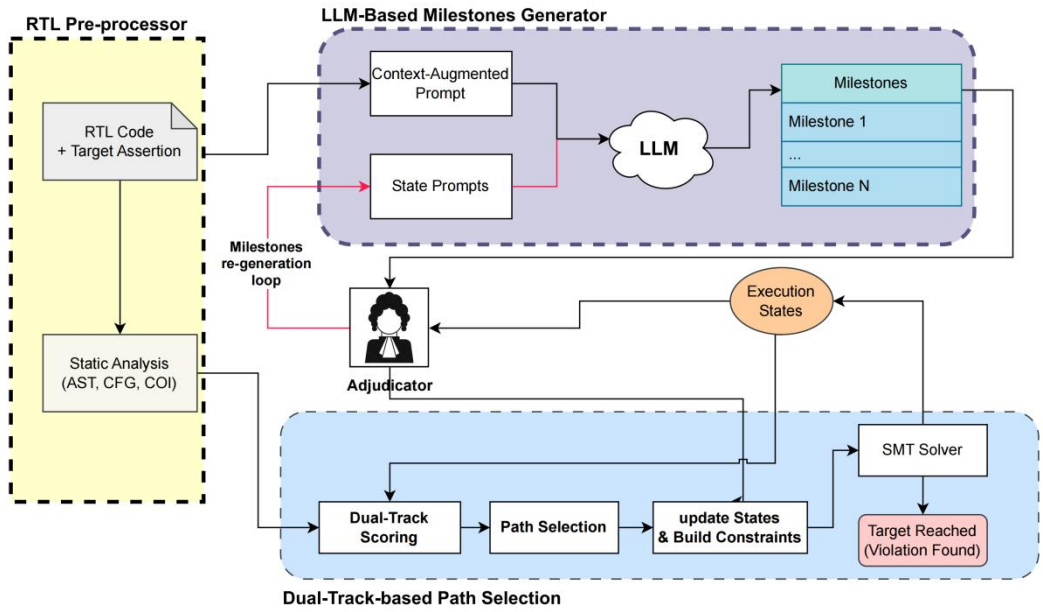


图 4-3-1 里程碑引导的符号执行验证流程

4.3.1硬件设计的上下文提取与大模型提示构建

大语言模型在生成探索引导时，若缺乏具体的硬件约束，极易产生引用不存在的信号或提出物理上不可达状态的“幻觉”。因此，本研究在路径探索开始前，首先利用前端工具（如 PySlang）解析 RTL 源代码，提取抽象语法树（AST）并构建控制流图（CFG）。为了进一步降低大模型的上下文 Token 开销并消除无关硬件噪声，本研究以目标断言和内部状态更新信号为端点，执行了影响锥（Cone-of-Influence, COI）剪枝。

在构建大模型提示词时，本研究综合了剪枝后的 RTL 代码、目标断言以及 CFG，迫使大模型的推理过程与实际的硬件拓扑保持一致。大模型被要求以标准化的 JSON 格式输出里程碑序列，每个里程碑精确包含意图说明、SystemVerilog 目标条件以及关键的周期界限（cycle_bound）。此外，当底层探索失败时，本研究会捕获当前的活动路径约束与寄存器快照，生成状态提示（State Prompt）反馈给大模型，驱动其从当前真实状态重新生成可行的后续引导，实现闭环的探索优化。

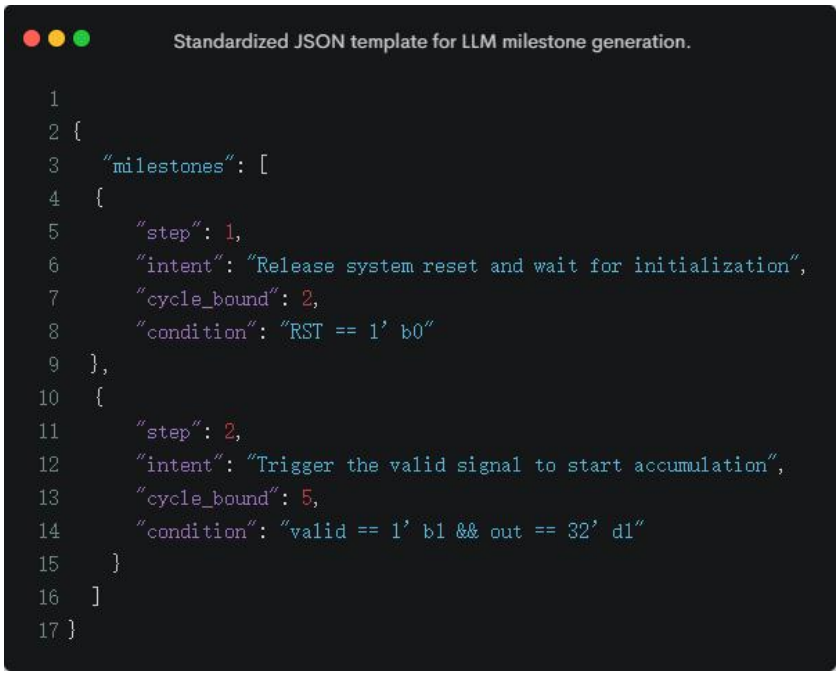


图 4-3-2 大模型生成的里程碑序列示例

4.3.2路径探索策略设计

在多周期跨度的符号执行中，由于各个里程碑之间存在时间间隔，如果缺乏细粒度的引导，符号执行引擎极易在间隔期退化为盲目的广度优先搜索（BFS），对大量无效路径进行探索。为了防止这种退化，本研究设计了双轨评分（Dual-Track Scoring）机制，对优先队列中的符号状态进行排序，双轨评分函数$f(n)$如下：

$$f(n) = \alpha * R(n) + C(n) + min(D(n), D_{max})$$

具体而言，双轨评分包含全局里程碑追踪与局部梯度追踪两个维度。

1. 全局里程碑追踪：R(n) 衡量剩余里程碑的数量，C(n) 记录经过的时钟周期数。高权重的 α 确保推进里程碑序列是最高优先级。

2. 局部梯度追踪：当跨越周期的里程碑进展停滞时，轻量级的距离度量 D(n) 将发挥作用。它基于 CFG 计算当前状态到目标条件的结构距离，提供局部梯度引导。

系统采用严格的分支策略，在条件分支处仅探索具有最佳 $D(n)$ 得分的“首选路径”，挂起其他合法分支，从而将昂贵的 SMT 求解资源集中于最直接的路径。

4.3.3面向大模型幻觉的容错裁决机制

由于大模型生成的里程碑本质上具有概率性，如果符号执行引擎将存在时序错位或逻辑错误的里程碑硬性注入路径约束，将导致 SMT 求解器频繁返回不可满足（UNSAT），使得验证过程陷入死锁。为解决这一问题，本研究在执行引擎内部设计了容错裁决器，实施三种动态纠偏机制：

（1）滑动窗口前瞻（Sliding-Window Lookahead）：引擎采用非阻塞的方式评估未来的目标窗口。如果发现序列中未来的某个里程碑已经满足，引擎将自动跳过中间由于大模型幻觉导致的不可达里程碑，实现局部的执行恢复。

（2）超时驱动重规划（Timeout-Driven Re-Generation）：本研究利用大模型生成的周期界限作为动态的超时阈值。如果在规定的周期数附加一定的容忍度后，状态仍未取得进展，裁决器将判定当前里程碑序列不可达，立即中止该停滞分支，并将当前状态反馈给大模型进行全局重新规划。

（3）急切目标评估（Eager Target Evaluation）：独立于任何中间里程碑，引擎在每一时钟周期的推进中都急切地评估最终的全局漏洞断言。这一保底机制确保了无论大模型的引导是否存在偏差，只要漏洞在物理状态上变为可达，系统即可第一时间截获并生成反例，保证了验证结果的严谨性。

4.3.4 实验设计与效果评估

本研究将 MileSE 框架实现为一个基于 Python 的定制符号执行引擎，结合 Z3 SMT 求解器，在 HACK@DAC 18/19/21 及开源 RISC 处理器核心 OR1200 等四个代表性多周期 RTL 安全基准测试集上进行了评估。本研究将业界的硬件符号执行引擎 Sylvia 作为基线进行了对比测试。

实验结果显示（表 3），本研究的方法在发现深层漏洞的效率与速度上均取得了显著提升。在总计 118 个验证属性中，本研究的方法成功检测出 113 个（基线 Sylvia 为 96 个），并在 OR1200 上实现了 100%的检测率且零超时。在执行时

间上，本研究的方法平均获得了 1.79 倍的加速，HACK@DAC 18 上的峰值加速达到 2.1 倍。

Benchmark	Sylvia(SOTA)			MileSE		
	#Viol	Timeout	Avg-t	#V	Timeout	Avg-t
HACK@DAC18(19)	12	7	50.2	**16**	**3**	**23.9**
HACK@DAC19(11)	7	4	76.3	**11**	**0**	**47.3**
HACK@DAC21(17)	13	4	113.4	**15**	**2**	**80.3**
OR1200(71)	64	7	39.4	**71**	**0**	**19.2**

表 3　与 Sylvia 的 bug 发现数量以及平均验证时间对比

此外，本研究针对裁决器的容错机制进行了消融实验。结果表明（图 4-3-4），“急切目标评估”（Eager）机制能够在单周期竞争条件等漏洞刚触发时立即捕获，大幅减少了 SMT 查询次数；而“滑动窗口前瞻”机制（Sliding）则有效避免了由不可达里程碑造成的超时现象。当这两种机制结合使用时，协同效应极其显著（如在 HACK@DAC 19 中，SMT 查询次数下降了约 85%），证明了容错裁决器在抑制多余约束注入、降低求解器负担以及抵御大模型幻觉方面的关键作用。

Benchmark	Config	#Viol	Timeout	Avg-t(s)	Avg-SMT
HACK@DAC18(19)	Baseline	12	3	29.6	3927
	+Eager	14	3	23.4	2042
	+Sliding	12	3	29.5	3287
	Full	14	3	23.9	2254
HACK@DAC19(11)	Baseline	10	1	104.3	5064
	+Eager	10	1	76.4	3674
	+Sliding	11	0	87.5	4508
	Full	11	0	47.3	760
HACK@DAC21(17)	Baseline	15	2	100.5	10526
	+Eager	15	2	83.8	6536
	+Sliding	15	2	101.1	12185
	Full	15	2	80.3	5978
OR1200(71)	Baseline	69	2	28.2	4892
	+Eager	70	1	22.3	3658
	+Sliding	70	1	21.2	2204
	Full	71	0	19.2	2287

表 4 基于里程碑的符号执行消融实验

5.研究工作计划与进度安排

在博士学习期间，本人期望能够设计并实现面向硬件验证的符号执行技术路径探索优化技术，具体包括：

1. 针对阻塞赋值中的依赖分析进行单周期路径探索的路径优化：对于阻塞赋值，结合其当周期立即生效且代码按顺序执行的特点，通过分析代码中描述组合逻辑电路的控制流程图和不同分支的依赖关系，确定完全独立的电路分支，在符号执行过程中根据上述信息避免重复访问已达状态，缓解单周期路径爆炸问题。目前论文已经被录用。
2. 针对非阻塞赋值中对寄存器值预测进行多周期路径探索的路径优化：对于非阻塞赋值，结合其在指定周期数后必定生效的特性，通过预测相关变量在未来周期的状态，提前加入路径约束中进行求解，提前过滤掉矛盾路径，避免之后周期不必要的路径访问，缓解多周期的路径爆炸问题。此研究基本功能已经完成，正在进行实验。
3. 里程碑引导的多周期 RTL 符号执行验证方法：利用大语言模型（LLM）生成高层语义里程碑来指导底层的符号执行路径探索，提出三个机制应对大模型固有的幻觉问题而导致的盲目路径探索，从而解决复杂硬件验证中面临的时间路径爆炸（Temporal Path Explosion）问题。目前论文已经完成，正在审稿过程中。

为了实现以上计划，本人的研究工作安排为：

1) 非阻塞赋值中对寄存器值预测进行多周期路径探索的路径优化（2 个月）
   a) 进行实验，完成论文撰写与投递（2 个月）
2) 毕业论文（3 个月）
   a) 总结三个研究点，完成博士论文，准备毕业答辩（3 个月）

6.已取得研究成果列表

已发表论文：

Jiahui Liang, Qiusong Yang*, and Yiwei Ci. Dataflow-based path pruning in symbolic execution for hardware verification[CCF-C]

在投论文：

Jiahui Liang, Qiusong Yang, Yiwei CiMileSE: Milestone-Guided Symbolic Execution for Multi-Cycle RTL Verification.[CCF-B] (ICCD-2026)

已授权专利：

《一种处理器微架构设计验证方法》。

发明人：梁佳辉，李亮　　申请号：2019106758398 发文序号：2022022201836200

已参与课题：

2024 年 1 月——至今，参加国家自然科学基金面上项目，资助号：62372438

2023 年 11 月——至今，参加中国科学院软件研究所基础研究项目，资助号：ISCAS-JCZD-202307

2020 年 1 月——2021 年 12 月，参加 GG 项目“新一代 XXXXXXXXXXXCPU”

2018 年 9 月——2019 年 7 月，参加中国科学院战略性先导科技专项 A 类，XDA-Y01-01，“关键软硬件系统安全性评估核心技术研发及验证”

7.参考文献

[1]. Shinya Takamaeda-Yamazaki: Pyverilog: A Python-based Hardware Design Processing Toolkit for Verilog HDL, 11th International Symposium on Applied

Reconfigurable Computing (ARC2015) (Poster), Lecture Notes in Computer Science, Vol.9040/2015, pp.451-460, April 2015.

[2]. N. Bruns, V. Herdt, and R. Drechsler: Processor Verification using Symbolic Execution: A RISC-V Case-Study (2023).

[3]. F. Fowze, M. Choudhury, and D. Forte, "EISec: Exhaustive Information Flow Security of Hardware Intellectual Property Utilizing Symbolic Execution," in Asian Hardware Oriented Security and Trust Symposium (AsianHOST)

[4]. R. Zhang, C. Deutschbein, P. Huang, and C. Sturton, "End-to-End Automated Exploit Generation for Validating the Security of Processor Designs," in Proceedings of the International Symposium on Microarchitecture (MICRO). IEEE/ACM, 2018.

[5]. C. Cadar, D. Dunbar, and D. Engler: KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In: OSDI, pp. 209–224 (2008)

[6]. Qiuping Yi, Zijiang Yang, Shengjian Guo, Chao Wang, Jian Liu, and Chen Zhao. 2018. Eliminating Path Redundancy via Postconditioned Symbolic Execution. IEEE Trans. Software Eng. 44, 1 (2018), 2543. https://doi.org/10.1109/TSE.2017.2659751

[7]. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) CONFERENCE 2016, LNCS, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). https://doi.org/10.10007/1234567890

[8]. Shinya Takamaeda-Yamazaki: Pyverilog: A Python-based Hardware Design Processing Toolkit for Verilog HDL, 11th International Symposium on Applied Reconfigurable Computing (ARC2015) (Poster), Lecture Notes in Computer Science, Vol.9040/2015, pp.451-460, April 2015.

[9]. L. Shen, D. Mu, G. Cao, M. Qin, J. Blackstone, and R. Kastner, Symbolic Execution Based Test-patterns Generation Algorithm for Hardware Trojan Detection, Comput. Secur., vol. 78, pp. 267280, 2018.

[10]. Ahmed, A., & Mishra, P. QUEBS: Qualifying Event Based Search in Concolic Testing for Validation of RTL Models.2017 IEEE International Conference on Computer Design (ICCD).2017

[11]. Lyu, Y., & Mishra, P. Scalable Concolic Testing of RTL Models. IEEE Transactions on Computers.2021; 70. https://doi.org/10.1109/TC.2020.2997644.

[12]. Meng, X., Kundu, S., Kanuparthi, A., & Basu, K. RTL-ConTest: Concolic Testing on RTL for Detecting Security Vulnerabilities.IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems.2021; 41. https://doi.org/10.1109/TCAD.2021.3066560.

[13]. Ahmed, A., Farahmandi, F., & Mishra, P. Directed test generation using concolic testing on RTL models. 2018 Design, Automation & Test in Europe Conference & Exhibition (DATE).2018 https://doi.org/10.23919/DATE.2018.8342260.

[14]. Dai, R., & Yavuz, T. Detecting Hardware Trojans using Model Guided Symbolic Execution.Proceedings of the Great Lakes Symposium on VLSI 2024.2024 https://doi.org/10.1145/3649476.3658782.

[15]. Jingxuan He, Gishor Sivanrupan, Petar Tsankov, and Martin Vechev. 2021. Learning to Explore Paths for Symbolic Execution. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS'21). Association for Computing Machinery, New York, NY, USA, 25262540. https://doi.org/10.1145/3460120.3484813

[16]. Bruns, N., Herdt, V., & Drechsler, R. (2023). Processor Verification using Symbolic Execution: A RISC-V Case-Study. 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE). Presented at the 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE), Antwerp, Belgium. https://doi.org/10.23919/date56975.2023.10137202

[17]. Fowze, F., Choudhury, M., & Forte, D. (2022). EISec: Exhaustive Information Flow Security of Hardware Intellectual Property Utilizing Symbolic Execution.

[18]. Witharana, H., Lyu, Y., & Mishra, P. (2021). Directed Test Generation for Activation of Security Assertions in RTL Models. ACM Transactions on Design Automation of Electronic Systems, 128. https://doi.org/10.1145/3441297

[19]. R. Zhang, C. Deutschbein, P. Huang, and C. Sturton, End-to-End Automated Exploit Generation for Validating the Security of Processor Designs, in Proceedings of the International Symposium on Microarchitecture (MICRO). IEEE/ACM, 2018.

[20]. Kaki Ryan, Matthew Gregoire, and Cynthia Sturton. 2023. SEIF: Augmented Symbolic Execution for Information Flow in Hardware Designs. In Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP '23). Association for Computing Machinery, New York, NY, USA, 19. https://doi.org/10.1145/3623652.3623666

[21]. https://github.com/google/AFL

[22]. https://verilator.org

[23]. https://github.com/steveicarus/iverilog

[24]. Majumdar, R., & Xu, R.-G. (2009). Reducing Test Inputs Using Information Partitions. Springer Berlin Heidelberg eBooks,Springer Berlin Heidelberg eBooks.

[25] rinivas Nidhra and Jagruthi Dondeti. Black box and white box testing techniques-a literature review. International Journal of Embedded Systems and Applications (IJESA), 2(2):29–50, 2012.

[26] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. Klee: unassisted and automatic generation of highcoverage tests for complex systems programs. In OSDI, volume 8, pages 209–224, 2008.

[27] You Li, Zhendong Su, Linzhang Wang, and Xuandong Li. Steering symbolic execution to less traveled paths. ACM SigPlan Notices, 48(10):19–32, 2013.

[28] Kasper Luckow, Corina S P as areanu, and Willem Visser. Monte carlo tree search for finding costly paths in programs. In Software Engineering and Formal Methods:

16th International Conference, SEFM 2018, Held as Part of STAF 2018, Toulouse, France, June 27–29, 2018, Proceedings 16, pages 123–138. Springer, 2018.

[29] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya

Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. Large language models for software engineering: Survey and open problems. arXiv preprint arXiv:2310.03533, 2023.

[30] Ipek Ozkaya. Application of large language models to software engineering tasks: Opportunities, risks, and implications. IEEE Software, 40(3):4–8, 2023.

[31] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

[32] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, R emi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. Science, 378(6624):1092–1097, 2022.

[33] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, J er emy Rapin, et al. Code llama:
Open foundation models for code. arXiv preprint arXiv:2308.12950, 2023.

[34] Qiuping Yi, Yifan Yu, and Guowei Yang. 2024. Compatible Branch Coverage Driven Symbolic Execution for Efficient Bug Finding. Proc. ACM Program. Lang. 8, PLDI, Article 213 (June 2024), 23 pages. https://doi.org/10.1145/3656443

[35] Haijun Wang, Ting Liu, Xiaohong Guan, Chao Shen, Qinghua Zheng, and Zijiang Yang. 2016. Dependence guided symbolic execution. *IEEE Transactions on Software Engineering 43, 3 (2016), 252–271.* https://doi.org/10.1109/TSE.2016.2584063

[36] J. Xu, J. Xu, T. Chen, and X. Ma, “Symbolic Execution with Test Cases Generated by Large Language Models,” in 2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS). Cambridge, United Kingdom: IEEE, Jul. 2024, pp. 228 – 237.

[37] H. Tu, S. Lee, Y. Li, P. Chen, L. Jiang, and M. Böhme, Cottontail: Large Language Model-Driven Concolic Execution for Highly Structured Test Input Generation, Oct. 2025.

[38] W. Wang, K. Liu, A. R. Chen, G. Li, Z. Jin, G. Huang, and L. Ma,Python Symbolic Execution with LLM-powered Code Generation, Sep. 2024.

[39] Z. Luo, H. Zhao, D. Wolff, C. Cadar, and A. Roychoudhury, Agentic Concolic Execution.

[40] Y. Tan, X. Meng, and Y. Lyu, LLM-assisted Path Exploration for RTL Verification, in 2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD). Santa Cruz, CA, USA: IEEE, Sep. 2025, pp. 1 – 7.

[41] S. Xia, M. He, S. Shao, T. Yu, Y. Zhang, and L. Song, SymGPT: Auditing Smart Contracts via Combining Symbolic Execution with Large Language Models, Feb. 2025.

[42] P. Srikumar, Fast and wrong: The case for formally specifying hardware with LLMs.

[43] R. Feng, V. Gupta, V. Patel, V. R. Ernampati, and S. Saha, Can Large Language Models Simulate Symbolic Execution Output Like KLEE? Nov. 2025.

[44] F. Xu, Z. Wu, Q. Sun, S. Ren, F. Yuan, S. Yuan, Q. Lin, Y. Qiao, and J. Liu, Symbol-LLM: Towards Foundational Symbol-centric Interface For Large Language Models, Feb. 202